

ALGORITHMS FOR DATASET PREP, HYBRID MODELS AND THEIR EXPLAINABILITY

Table of Contents

ALGORITHMS FOR DATASET PREP, HYBRID MODELS AND THEIR EXPLAINABILITY 1

ALGORITHM 1 Dataset Preparation..... 2

Algorithm 1A – Hybrid A (PFD-GSTE Variant A) Forward Pass..... 7

Algorithm 1B – Hybrid B (PFD-GSTE Variant B) Forward Pass 8

Algorithm XAI-2A – XAI for Hybrid A (PFD-GSTE Variant A) 10

Algorithm XAI-2B – XAI for Hybrid B (PFD-GSTE Variant B) 12

****Notation:** $x \leftarrow y$ means assign the value of y into variable x (i.e., store y in x). It is pseudocode assignment, like $x = y$ in code, I followed this style since papers such as Krishnan et al. did theirs in the same way (Krishnan et al., 2024).

ALGORITHM 1 Dataset Preparation

INPUT:

- RAW_ROOT containing two source splits: {Training, Testing}
- CLASSES = {glioma, meningioma, pituitary, notumor}
- IMG_SIZE = (224, 224)
- VAL_FRACTION = 0.20
- RANDOM_STATE = 42
- ALLOWED_EXTS = {jpg, jpeg, png, bmp, tif, tiff}
- PREFER_TESTING_ON_CROSS_SPLIT_DUPLICATES = True
- BACKGROUND_INTENSITY_THRESHOLD = 5
- CROP_MARGIN = 10
- RESAMPLE = LANCZOS (Pillow-compatible selection)

OUTPUT:

- Processed 224×224 RGB images (tight-cropped)
- Split tables for train / val / test (image_path, class)
- Audit tables (raw class counts, duplicate report, raw-image stats and quality flags, split summary)

BEGIN

```
# 1) Collect raw file records from Kaggle-style folders
records ← empty list
FOR each source_split IN {Training, Testing} DO
  FOR each cls IN CLASSES DO
    folder ← RAW_ROOT/source_split/cls
    IF folder does not exist THEN
      CONTINUE
    ENDIF

    FOR each item p IN folder DO
      IF p is not a file THEN CONTINUE ENDIF
      IF p.name starts with "." THEN CONTINUE ENDIF
      IF extension(p) ∉ ALLOWED_EXTS THEN CONTINUE ENDIF

      APPEND to records:
        (orig_path = p,
         class = cls,
         source_split = lowercase(source_split)) # "training" or "testing"
    ENDFOR
  ENDFOR
ENDFOR
df_raw ← DataFrame(records)

# 2) Summarise raw class counts by Kaggle source split (audit only)
Save_Raw_Class_Counts(df_raw)

# 3) Leakage-safe exact deduplication using SHA1(file_bytes)
(df_dedup, dups_df) ← Drop_Duplicates_LeakageSafe(df_raw,
                                                    prefer_testing = PREFER_TESTING_ON_CROSS_SPLIT_DUPLICATES)
Save_Duplicate_Report(dups_df)

# 4) Raw-image audit (geometry + intensity) with flags ONLY (no dropping)
stats_df ← Analyse_Raw_Images(df_dedup)
Save_Raw_Audit_Tables(stats_df)

# 5) Kaggle-aligned split creation
(df_train, df_val, df_test) ← Make_Splits_KaggleAligned(df_dedup,
                                                         val_fraction = VAL_FRACTION,
                                                         random_state = RANDOM_STATE)

# 6) Prepare output structure for processed images
Prepare_Processed_Directories(CLASSES, splits = {train, val, test})
```

```

# 7) Process images for each split (robust: failures logged, pipeline continues)
Resize_And_Copy(df_train, "train", IMG_SIZE, RESAMPLE,
                BACKGROUND_INTENSITY_THRESHOLD, CROP_MARGIN)
Resize_And_Copy(df_val, "val", IMG_SIZE, RESAMPLE,
                BACKGROUND_INTENSITY_THRESHOLD, CROP_MARGIN)
Resize_And_Copy(df_test, "test", IMG_SIZE, RESAMPLE,
                BACKGROUND_INTENSITY_THRESHOLD, CROP_MARGIN)

# 8) Build split CSV-style tables pointing to processed images + class labels
Save_Split_Tables(df_train, df_val, df_test)

# 9) Save split summary counts per class (audit)
Save_Split_Summary(df_train, df_val, df_test)
END

```

SUBROUTINE Drop_Duplicates_LeakageSafe(df, prefer_testing)

INPUT: df with columns (orig_path, class, source_split)

OUTPUT: (dedup_df, dups_df)

BEGIN

IF df is empty THEN

RETURN (copy(df), empty DataFrame)

ENDIF

Compute SHA1 per file (byte-for-byte identity)

FOR each row r IN df DO

r.sha1 ← SHA1_Of_File(r.orig_path)

ENDFOR

dup_mask ← rows that have duplicated sha1 (mark ALL members of each dup group)

dups_df ← df where dup_mask is True

IF dups_df is empty THEN

RETURN (df reset_index, dups_df)

ENDIF

keep_indices ← empty list

FOR each sha1 group G IN groupby(dups_df, sha1) DO

Deterministic order

G ← sort G by (source_split, orig_path)

has_testing ← any row in G where source_split = "testing"

has_training ← any row in G where source_split = "training"

IF prefer_testing AND has_testing AND has_training THEN

keep_row ← first row of (rows in G with source_split="testing" sorted by orig_path)

ELSE

keep_row ← first row of (G sorted by orig_path)

ENDIF

APPEND keep_row.original_index TO keep_indices

ENDFOR

non_dup_df ← df where dup_mask is False

kept_from_dups_df ← df rows at keep_indices

dedup_df ← concat(non_dup_df, kept_from_dups_df) and reset_index

Mark kept/dropped for audit

dups_df.kept ← False

FOR each idx IN keep_indices DO

set dups_df.kept = True where dups_df.original_index == idx

ENDFOR

dups_df.kept_path_for_group ← (for each sha1, the orig_path where kept=True)

```
    RETURN (dedup_df, dups_df)
END
```

```
SUBROUTINE Analyse_Raw_Images(df)
```

```
INPUT: df with (orig_path, class, source_split, sha1)
```

```
OUTPUT: stats_df with geometry + intensity + flags
```

```
BEGIN
```

```
    records ← empty list
```

```
    FOR each row r IN df DO
```

```
        failed ← False
```

```
        TRY
```

```
            img ← Open_Image(r.orig_path)
```

```
            img ← EXIF_Transpose(img)
```

```
            Force_Decompose(img)
```

```
            (width, height) ← img.size
```

```
            aspect_ratio ← width / height
```

```
            gray ← Convert_To_Grayscale(img)
```

```
            arr ← To_Array(gray)
```

```
            mean_intensity ← mean(arr)
```

```
            std_intensity ← std(arr)
```

```
            min_intensity ← min(arr)
```

```
            max_intensity ← max(arr)
```

```
        CATCH any error
```

```
            failed ← True
```

```
            width, height, aspect_ratio ← null
```

```
            mean_intensity, std_intensity, min_intensity, max_intensity ← null
```

```
        ENDTRY
```

```
    APPEND to records:
```

```
        (orig_path, class, source_split, sha1,
```

```
        width, height, aspect_ratio,
```

```
        mean_intensity, std_intensity, min_intensity, max_intensity,
```

```
        failed)
```

```
    ENDFOR
```

```
stats_df ← DataFrame(records)
```

```
not_failed ← NOT stats_df.failed
```

```
stats_df.too_dark ← not_failed AND (mean_intensity < 15) AND (max_intensity < 60)
```

```
stats_df.too_bright ← not_failed AND (mean_intensity > 240) AND (min_intensity > 200)
```

```
stats_df.low_contrast ← not_failed AND (std_intensity < 5)
```

```
stats_df.suspect ← (too_dark OR too_bright OR low_contrast OR failed)
```

```
RETURN stats_df
```

```
END
```

```
SUBROUTINE Make_Splits_KaggleAligned(df, val_fraction, random_state)
```

```
INPUT: df with (orig_path, class, source_split)
```

```
OUTPUT: (df_train, df_val, df_test) with split labels {train, val, test}
```

```
BEGIN
```

```
    IF df is empty THEN
```

```
        RAISE error "Cannot split empty dataset"
```

```
    ENDIF
```

```
    df_train_source ← rows where source_split = "training"
```

```
    df_test_source ← rows where source_split = "testing"
```

```
    IF df_train_source is empty THEN
```

```

    RAISE error "No training images after deduplication"
ENDIF
# df_test_source may be empty; allowed (warn in implementation)

X ← df_train_source.orig_path
y ← df_train_source.class

TRY
    (X_train, X_val, y_train, y_val) ← TrainTestSplit(
        X, y,
        test_size = val_fraction,
        stratify = y,
        random_state = random_state
    )
CATCH any error
    (X_train, X_val, y_train, y_val) ← TrainTestSplit(
        X, y,
        test_size = val_fraction,
        random_state = random_state
    )
ENDTRY

df_train ← DataFrame(orig_path=X_train, class=y_train, split="train")
df_val  ← DataFrame(orig_path=X_val, class=y_val, split="val")
df_test ← DataFrame(orig_path=df_test_source.orig_path,
                    class=df_test_source.class,
                    split="test")

RETURN (df_train, df_val, df_test)
END

```

```

SUBROUTINE Tight_Crop_To_Brain(img, threshold, margin)
INPUT: PIL image img (RGB or convertible), threshold, margin
OUTPUT: cropped image (or original if no foreground found)
BEGIN
    gray ← Convert_To_Grayscale(img)
    arr ← To_UInt8_Array(gray)
    mask ← arr > threshold

    IF mask has no True pixels THEN
        RETURN img
    ENDIF

    (ys, xs) ← coordinates where mask is True

    y_min ← max(min(ys) - margin, 0)
    y_max ← min(max(ys) + 1 + margin, height(arr))
    x_min ← max(min(xs) - margin, 0)
    x_max ← min(max(xs) + 1 + margin, width(arr))

    RETURN Crop(img, left=x_min, bexpx=y_min, right=x_max, lower=y_max)
END

```

```

SUBROUTINE Resize_And_Copy(df_split, split_name, IMG_SIZE, RESAMPLE, threshold, margin)
INPUT: split table df_split, split_name ∈ {train,val,test}, size, resampler, crop params
OUTPUT: processed images written (failures logged; continue)
BEGIN
    FOR each row r IN df_split DO
        src_path ← r.orig_path
        cls ← r.class
        out_name ← SHA1_Of_File(src_path) + Lowercase_Extension(src_path)
        dst_path ← Destination_For(split_name, cls, out_name)
    END FOR
END

```

```
IF dst_path already exists THEN
  CONTINUE
ENDIF

TRY
  img ← Open_Image(src_path)
  img ← EXIF_Transpose(img)
  img ← Convert_To_RGB(img)
  img ← Tight_Crop_To_Brain(img, threshold, margin)
  img ← Resize(img, IMG_SIZE, resample=RESAMPLE)
  Save_Image(img, dst_path)
CATCH any error
  Log("Failed to process", src_path)
ENDTRY
ENDFOR
END
```

Algorithm 1A – Hybrid A (PFD-GSTE Variant A) Forward Pass

Input: image batch x , flag return_xai

Output: logits, and optionally {"mask": mask_img, "attn": attn_list}

1. $\text{feat} \leftarrow \text{CNN}(x)$ (ResNet50V2 feature map, e.g., $(B, 2048, 7, 7)$)
2. $\text{z_cnn} \leftarrow \text{AvgPool}(\text{feat})$
3. $\text{z_cnn} \leftarrow \text{Flatten}(\text{z_cnn})$
4. $\text{z_cnn} \leftarrow \text{ReLU}(\text{Linear_cnn_proj}(\text{z_cnn}))$
5. $(\text{feat_path}, \text{mask_feat}) \leftarrow \text{PFD}(\text{feat})$
 - $\text{mask_feat} \leftarrow \text{Sigmoid}(\text{Conv1x1}(\text{feat}))$
 - $\text{feat_path} \leftarrow \text{feat} * \text{mask_feat}$
6. $\text{mask_img} \leftarrow \text{BilinearUpsample}(\text{mask_feat}, \text{size} = \text{input_image_size})$ (only for visualisation/XAI)
7. $\text{token_sets} \leftarrow \text{empty list}$
8. **For each** ($\text{rot_idx}, k$) **in** $\text{enumerate}(\text{rotations})$ **do**
 - 8.1 $\text{f_r} \leftarrow \text{rot90}(\text{feat_path}, k)$
 - 8.2 $\text{m_r} \leftarrow \text{rot90}(\text{mask_feat}, k)$
 - 8.3 $(\text{tokens}, \text{ht}, \text{wt}) \leftarrow \text{FeatureTokenEmbed}(\text{f_r})$
 - 8.4 $\text{alpha} \leftarrow \text{Flatten}(\text{m_r})$ as $(B, \text{ht} * \text{wt}, 1)$
 - 8.5 $\text{alpha} \leftarrow \text{alpha} / (\text{mean}(\text{alpha over tokens}) + 1e-6)$
 - 8.6 $\text{tokens} \leftarrow \text{tokens} * \text{alpha}$ (GSTE-A: static guidance weighting)
 - 8.7 $\text{tokens} \leftarrow \text{PosRotEmbed}(\text{tokens}, \text{ht}, \text{wt}, \text{rot_idx})$
 - 8.8 **append tokens to token_sets**
- Endfor**
9. $\text{Tavg} \leftarrow \text{Mean}(\text{token_sets over rotations})$
10. $(\text{Tenc}, \text{attn_list}) \leftarrow \text{RViTEncoder}(\text{Tavg}, \text{return_attn} = \text{return_xai})$
11. $\text{z_vit} \leftarrow \text{Mean}(\text{Tenc over tokens})$
12. $\text{z_vit} \leftarrow \text{ReLU}(\text{Linear_vit_proj}(\text{z_vit}))$
13. $\text{z} \leftarrow \text{Concat}(\text{z_cnn}, \text{z_vit})$
14. $\text{h} \leftarrow \text{ReLU}(\text{Linear_fuse_fc}(\text{z}))$
15. $\text{h} \leftarrow \text{Dropout}(\text{h})$
16. $\text{logits} \leftarrow \text{Linear_out}(\text{h})$
17. **If** return_xai **then**
 - Return** logits, {"mask": mask_img, "attn": attn_list}
 - Else**
 - Return** logits, None
- Endif**

Algorithm 1B – Hybrid B (PFD-GSTE Variant B) Forward Pass

Input: image batch x , flag `return_xai`

Output: logits, and optionally `{"attn": attn_list, "mask": mask_img, "gste_side": dyn_side}`

1. $feat \leftarrow CNN(x)$ (ResNet50V2 feature map, e.g., $(B, C, 7, 7)$)
2. **If** `use_pfd_gste == True` **then**
 - 2.1 $(feat_path, mask_feat) \leftarrow PFD(feat)$
 - 2.2 $feat_for_cnn \leftarrow feat_path$
 - 2.3 $mask_img \leftarrow BilinearUpsample(mask_feat, size = input_image_size)$**Else**
 - 2.4 $feat_for_cnn \leftarrow feat$
 - 2.5 $mask_img \leftarrow None$**Endif**
3. $cache_last_cnn_feat \leftarrow feat_for_cnn$
4. $cache_last_cnn_mask_img \leftarrow mask_img$
5. **CNN descriptor head**
 - 5.1 $z_cnn \leftarrow AvgPool(feat_for_cnn)$
 - 5.2 $z_cnn \leftarrow Flatten(z_cnn)$
 - 5.3 $z_cnn \leftarrow Dropout(z_cnn)$
 - 5.4 $z_cnn \leftarrow Linear_cnn_proj(z_cnn)$
 - 5.5 $z_cnn \leftarrow BatchNorm(z_cnn)$
 - 5.6 $z_cnn \leftarrow ReLU(z_cnn)$
6. **Choose dynamic token grid side (GSTE decision)****If** `use_pfd_gste == True` **then**
 - 6.1 $alpha0 \leftarrow AdaptiveAvgPool(mask_img, output=(g, g))$ ($g = base_grid = img_size // patch_size$)
 - 6.2 $alpha0 \leftarrow alpha0 / (mean(alpha0 \text{ over grid}) + 1e-6)$
 - 6.3 $dyn_side \leftarrow choose_dynamic_side(alpha0)$ where:
 - $s \leftarrow mean(std(alpha0 \text{ over grid cells}))$
 - **If** $s \leq gste_std_flat$ **then** $dyn_side = g$
 - **Else**
 - $t \leftarrow clamp((s - gste_std_flat) / (gste_std_full - gste_std_flat), 0, 1)$
 - $shrink \leftarrow gste_max_shrink * t$
 - $target \leftarrow round(g * (1 - shrink))$
 - $dyn_side \leftarrow clamp(target, gste_min_side, g)$**Else**
 - 6.4 $dyn_side \leftarrow g$**Endif**
 7. $token_sets \leftarrow \text{empty list}$
 8. **For each** k **in** `rotations` **do**
 - 8.1 $rot_id \leftarrow k \bmod 4$
 - 8.2 $x_r \leftarrow rot90(x, rot_id)$
 - 8.3 $(tokens_base, ht, wt) \leftarrow ImagePatchEmbed(x_r)$ (tokens on $g \times g$ grid)
 - 8.4 **If** `use_pfd_gste == True` **then**
 - 8.4.1 $m_r \leftarrow rot90(mask_img, rot_id)$
 - 8.4.2 $alpha_base \leftarrow AdaptiveAvgPool(m_r, output=(g, g))$
 - 8.4.3 $alpha_base \leftarrow alpha_base / (mean(alpha_base \text{ over grid}) + 1e-6)$
 - 8.4.4 $(tokens, ht, wt) \leftarrow GSTE_dynamic_tokens(tokens_base, alpha_base, dyn_side)$ where:
 - $reshape\ tokens_base \rightarrow (B, D, g, g)$
 - $multiply\ by\ alpha_base\ (ROI\ weighting)$
 - **If** $dyn_side < g$ **then** $weighted\ pool\ to\ (dyn_side, dyn_side)$ using (num/den)
 - $flatten\ back \rightarrow (B, dyn_side * dyn_side, D)$**Else**


```
8.5 tokens  $\leftarrow$  tokens_base  
Endif
```

```
8.6 tokens  $\leftarrow$  PosRotEmbed(tokens, ht, wt, rot_id)  
8.7 append tokens to token_sets  
Endfor
```

```
9. Tavg  $\leftarrow$  Mean(token_sets over rotations)  
10. (Tenc, attn_list)  $\leftarrow$  RViTEncoder(Tavg, return_attn = return_xai)  
11. z_vit  $\leftarrow$  Mean(Tenc over tokens)  
12. z_vit  $\leftarrow$  Linear_vit_proj(z_vit)  
13. z_vit  $\leftarrow$  ReLU(z_vit)  
14. z  $\leftarrow$  Concat(z_cnn, z_vit)  
15. h  $\leftarrow$  Linear_fuse_fc(z)  
16. h  $\leftarrow$  ReLU(h)  
17. h  $\leftarrow$  Dropout(h)  
18. logits  $\leftarrow$  Linear_out(h)  
19. If return_xai then  
    Return logits, {"attn": attn_list, "mask": mask_img, "gste_side": dyn_side}  
    Else  
    Return logits, None  
    Endif
```

Algorithm XAI-2A – XAI for Hybrid A (PFD-GSTE Variant A)

Input: --checkpoint, --image, --out_dir, --mc_samples, --target_class

Output: saved overlays (xai_gradcampp.png, xai_attn_rollout.png) and printed prediction summary

1. Parse CLI arguments: checkpoint, image, out_dir, mc_samples, target_class
2. device $\leftarrow$ "cuda" if available else "cpu"
3. Create out_dir if it does not exist
4. Load checkpoint ckpt $\leftarrow$ torch.load(checkpoint, map_location=device)
5. Read from checkpoint (with fallbacks):
 - o class_names $\leftarrow$ ckpt["class_names"] else ["glioma", "meningioma", "pituitary", "notumor"]
 - o mean $\leftarrow$ ckpt["mean"] else [0.485, 0.456, 0.406]
 - o std $\leftarrow$ ckpt["std"] else [0.229, 0.224, 0.225]
 - o cfg $\leftarrow$ ckpt["model_cfg"] else {}
6. Build Hybrid A model using cfg (with defaults) and set cnn_pretrained=False
7. Load weights:
 - o state $\leftarrow$ ckpt["model_state"] if exists else ckpt["state_dict"]
 - o If state missing $\rightarrow$ raise error
 - o model.load_state_dict(state)
8. model.eval()
9. Load image using PIL, convert to RGB, resize to (224,224)
10. Preprocess image:
 - o Convert to tensor in [0,1], permute to (3,H,W)
 - o Normalize using mean,std
 - o Add batch dimension $\rightarrow$ x shaped (1,3,224,224)
 - o Move x to device
11. Prepare a dictionary hook_cache $\leftarrow$ {}
12. Define **PFD forward hook** _pfd_hook(module, inputs, output):
 - o feat_path $\leftarrow$ output[0] (*PFD returns (feat_path, mask_feat)*)
 - o feat_path.retain_grad()
 - o hook_cache["cnn_feat"] $\leftarrow$ feat_path
13. Register hook: hook_handle $\leftarrow$ model.pfd.register_forward_hook(_pfd_hook)
14. Run forward with gradients enabled:
 - o with torch.enable_grad():
 - (logits, xai) $\leftarrow$ model(x, return_xai=True)
15. Remove hook: hook_handle.remove()
16. Compute prediction:
 - o prob $\leftarrow$ softmax(logits)[0]
 - o pred_idx $\leftarrow$ argmax(prob)
 - o conf $\leftarrow$ prob[pred_idx]
17. Choose explanation class:
 - o If target_class < 0 then target $\leftarrow$ pred_idx
 - o Else target $\leftarrow$ target_class
18. Backprop for Grad-CAM++:
 - o model.zero_grad(set_to_none=True)
 - o logits[0, target].backward(retain_graph=True)
 - o A $\leftarrow$ hook_cache["cnn_feat"]
 - o If A missing $\rightarrow$ raise error
19. Build **Grad-CAM++** heatmap:
 - o cam_small $\leftarrow$ gradcam_pp_from_activations(A, A.grad) (*feature-map resolution*)
 - o Upsample to (224,224) using bilinear interpolate $\rightarrow$ cam
 - o Convert cam to NumPy for plotting
20. Build **Attention Rollout** heatmap:
 - o attn_list $\leftarrow$ xai["attn"] if exists else xai["attn_list"]
 - o If missing $\rightarrow$ raise error
 - o attn_tok $\leftarrow$ attention_rollout(attn_list) (*returns ht×wt token map inferred from N*)
 - o Upsample attn_tok to (224,224) using bilinear interpolate $\rightarrow$ attn_img
 - o Convert attn_img to NumPy

21. MC Dropout uncertainty:

- with torch.no_grad(): (mu, var) ← model.mc_dropout_predict(x, mc_samples)
- Convert to NumPy
- mu_pred ← argmax(mu)
- mu_conf ← mu[mu_pred]
- mu_var ← var[mu_pred]

22. Save overlays:

- overlay_and_save(img_rgb, cam, out_dir/"xai_gradcampp.png", "Grad-CAM++ (target=...)")
- overlay_and_save(img_rgb, attn_img, out_dir/"xai_attn_rollout.png", "Attention Rollout (token relevance)")

23. Print:

- single-pass predicted class, confidence
- MC-dropout mean class, mean confidence, predictive variance
- saved file paths

Algorithm XAI-2B — XAI for Hybrid B (PFD-GSTE Variant B)

Input: --checkpoint, --image, --out_dir, --mc_samples, --target_class

Output: saved overlays (xai_gradcampp.png, xai_attn_rollout.png) and printed prediction summary

1. Parse CLI arguments: checkpoint, image, out_dir, mc_samples, target_class
2. Force matplotlib headless backend: `matplotlib.use("Agg")`
3. `device ← "cuda" if available else "cpu"`
4. Create out_dir if it does not exist
5. Load checkpoint `ckpt ← torch.load(checkpoint, map_location=device)`
6. Read from checkpoint (with fallbacks):
 - `class_names ← ckpt["class_names"]` else `["glioma", "meningioma", "pituitary", "notumor"]`
 - `mean ← ckpt["mean"]` else `[0.5, 0.5, 0.5]`
 - `std ← ckpt["std"]` else `[0.5, 0.5, 0.5]`
 - `cfg ← ckpt["model_cfg"]` else `{}`
7. Build Hybrid B model from cfg and set:
 - `cnn_pretrained=False`
 - `use_pfd_gste=True` (*critical: ensures mask + dynamic tokens exist*)
8. Load weights:
 - `state ← ckpt["model_state"]` if exists else `ckpt["state_dict"]`
 - If missing → raise error
 - `model.load_state_dict(state)`
9. `model.eval()`
10. Load image using PIL, convert to RGB, resize (224,224)
11. Preprocess image exactly like script:
 - Tensor in [0,1], permute to (3,H,W)
 - Normalize by mean,std
 - Add batch dim → (1,3,224,224)
 - Move to device
12. Prepare `hook_cache ← {}`
13. Define **PFD forward hook** `_pfd_hook(module, inputs, output)`:
 - Verify output is tuple/list
 - `feat_path ← output[0]`
 - `feat_path.retain_grad()`
 - `hook_cache["cnn_feat"] ← feat_path`
14. Register hook: `h ← model.pfd.register_forward_hook(_pfd_hook)`
15. **Try / Finally** (so hook is always removed):
 - Try**
 - 15.1 with `torch.enable_grad()`: `(logits, xai) ← model(x, return_xai=True)`
 - 15.2 If xai not dict → raise error
 - 15.3 If `xai["mask"]` is None → raise error (*script enforces PFD/GSTE active*)
 - 15.4 `prob ← softmax(logits)[0]`
 - 15.5 `pred_idx ← argmax(prob)` and `conf ← prob[pred_idx]`
 - 15.6 If `target_class < 0` then `target ← pred_idx` else `target ← target_class`
 - 15.7 `model.zero_grad(set_to_none=True)`
 - 15.8 `logits[0, target].backward()`
 - 15.9 `A ← hook_cache["cnn_feat"]`
 - 15.10 If A missing or A.grad missing → raise error
 - **Finally**
 - 15.11 `h.remove()`
16. Build **Grad-CAM++** heatmap:
 - `cam_small ← gradcam_pp_from_activations(A, A.grad)`
 - Upsample to (224,224) using bilinear interpolate → cam
 - Convert cam to NumPy
17. Build **Attention Rollout** heatmap (uses GSTE side if available):
 - `attn_list ← xai["attn"]` if exists else `xai["attn_list"]`
 - If missing → raise error
 - `gste_side ← xai.get("gste_side", None)`

- `attn_tok ← attention_rollout(attn_list, side=gste_side)`
 - If side is provided, script **requires** `side*side == N` or raises error
 - Upsample `attn_tok` to (224,224) → `attn_img`
 - Convert `attn_img` to NumPy
18. MC Dropout uncertainty:
- with `torch.no_grad()`: `(mu, var) ← model.mc_dropout_predict(x, mc_samples)`
 - Convert to NumPy
 - `mu_pred ← argmax(mu)`
 - `mu_conf ← mu[mu_pred]`
 - `mu_var ← var[mu_pred]`
19. Save overlays:
- `overlay_and_save(img_rgb, cam, out_dir/"xai_gradcampp.png", "Grad-CAM++ (PFD target=...)"`
 - `overlay_and_save(img_rgb, attn_img, out_dir/"xai_attn_rollout.png", "Attention Rollout")`
20. Print:
- single-pass predicted class, confidence
 - MC-dropout mean class, mean confidence, predictive variance
 - saved file paths